NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Applied Mathematics and Informatics"

Research Project Report on the Topic:

Personality-Aware Sequential Self-Attention Learning for Next Item Recommendations

Submitted by the Student:

group #БПМИ232, 3rd year of study Abramov Georgiy Evgenievich

Approved by the Project Supervisor:

Frolov Evgeniy Petrovich
Associate Professor
Faculty of Computer Science, HSE University

Co-supervisor:

Anton Lysenko Viktorovich
Expert
Faculty of Computer Science, HSE University

Moscow 2026

Contents

Annotation **3**

1 Introduction **4**

2 Related work **5**

2.1 Sequential self-attention recommendations 5

2.2 Introduction of a new dimension for rank score calculation 5

3 Methodology **7**

4 Experimental setup **7**

4.1 Datasets 7

4.2 Evaluation 8

5 Results **9**

5.1 Standard setting performance comparison 9

5.2 Session-aware setting performance comparison 10

5.3 Approaches for incorporating user embeddings for CP 11

5.4 Tucker decomposition 12

6 Conclusion **13**

References **14**

Annotation

The self-attention mechanism is a key approach for processing sequential data in machine learning. Its application in recommender systems has led to the emergence of new state-of-the-art models that predict next items for the user based on their history of previously interacted items. However, such algorithms often do not explicitly model the user and handle sequences of items from different users identically. In this work, we investigate different ways of incorporating user information into sequential recommendation models.

Аннотация

Механизм самовнимания является ключевым способом обработки последовательных данных в машинном обучении. Его применение в рекомендательных системах привело к появлению новых передовых моделей, которые по последовательности товаров предсказывают следующие предметы в истории пользователя. Однако, такие алгоритмы часто не учитывают пользователя в явном виде и обрабатывают последовательности товаров от разных пользователей одинаково. В данной работе исследуются различные способы включения знания о пользователе в работу последовательной модели для рекомендаций.

Keywords

Deep learning, Recommender systems, Self-attention, Transformers, Session-aware recommendation, Tensor decomposition

1 Introduction

Next item prediction recommender systems aim to forecast the next item for a user, given their history of interactions with items. Lately, more and more state-of-the-art models have adopted the *sequential* approach to processing such data, where items are treated explicitly as a sequence, and the order of interactions is important. This usually allows for better recommendations, since we also use the information about the time of interactions.

Since the introduction of transformers [21] for natural language processing (NLP) tasks, they have been successfully applied in other fields of machine learning, recommender systems being one of them. One of the most important works that accomplishes this is Self-Attentive Sequential Recommendation (SASRec) [11], inspired by Generative Pretrained Transformer (GPT) [18]. It functions similarly and adopts the next token prediction task from NLP as a next item prediction task for recommender systems. Effectiveness of the transformer architecture carried over from NLP to recommender systems, and SASRec became one of the state-of-the-art models for sequential next item recommendations. Thus, in this work we will focus on modifying this model.

To make recommendations personalized, some user modeling is required. One of the most popular approaches is to learn *user embeddings*. There are two main types of such embeddings: *explicit*, which represent user preferences learned from data (e.g., matrix factorization [7]), and *implicit*, which constructs an embedding representing user preferences from a history of previously interacted with items (e.g., GRU4Rec [8]).

SASRec achieves personalized recommendations by producing an implicit user embedding based on their history. In this work, we combine both explicit and implicit approaches to user modeling by introducing an explicit learned user embedding to the SASRec architecture. Thus, we allow the model to regularize sequence evolution with persistent user-level signals. This also leads to better personalization, since two different users with the same history will receive different recommendations, in contrast to standard SASRec.

Many item interactions can be viewed as a collection of sessions (e.g., several tracks listened to one after another in a music subscription service). If we do not have access to user identification (e.g., e-commerce recommendations where we have only the current shopping sequence), this constitutes a *session-based* setting. However, in cases where we are given the user identity, we can enrich the recommendations based on long-term user preferences learned from the past sessions – this is called the *session-aware* setting [15]. For our modified approach, we utilize the increased personalization further by considering the *session-aware* setting with explicit user embedding representing constant long-term user preferences.

The original SASRec architecture constructs the implicit user embedding as a last hidden state of the last item in the user history, similar to the last hidden state of the last token being used for next token prediction in [18]. Items are ranked for recommendation based on this embedding and learned item embeddings. In our work, we add a new dimension to the recommendation rank calculation – explicit user embedding – and explore several strategies for incorporating it.

The code and configurations for this paper can be found on GitHub[1].

2 Related work

2.1 Sequential self-attention recommendations

Transformer-based architectures (e.g., SASRec [11], BERT4Rec [19]) consistently tend to outperform other algorithms in the area of sequential recommendations. SASRec operates similarly to GPT; however, there are several key differences. Firstly, SASRec strictly shares the item input embeddings with the last classification layer, since in recommender systems we often work with large item catalogues, and most of the parameters are item embeddings. Secondly, SASRec is trained with the Binary Cross Entropy loss (BCE) with negative sampling instead of the full Cross Entropy loss (CE). This is another limitation due to the large item catalogue, since for CE we have to calculate a full $B \cdot L \cdot N$ logits tensor, where B is the batch size, L is the sequence length, and N is the size of the item catalogue. For BCE with negative sampling, we have to calculate only a $B \cdot L \cdot (N_{neg} + 1) \cdot D$ hidden states tensor, where N_{neg} is the number of negative elements sampled for each position in a sequence and D is the hidden dimension of the network. In the original paper, SASRec was trained with the BCE loss, while BERT4Rec was trained with the CE loss and showed better performance. Subsequent research [13] showed that this was the key reason for the differences in the quality of recommendations and not the bidirectional nature of the BERT4Rec model. This shows the importance of the loss function when training sequential transformer-based models in settings with large items catalogues. Informed by this, we choose the Scalable Cross Entropy (SCE) loss [16], which is optimized for large item catalogues and in some scenarios shows even better performance than CE.

2.2 Introduction of a new dimension for rank score calculation

The original SASRec work ranks items for recommendation based on the score calculated as $r_{tj} = h_t^\top i_j$ where $h_t \in \mathbb{R}^D$ is the hidden state of the sequence at the timestep t and $i_j \in \mathbb{R}^D$ is

[1] https://github.com/abramovgeorge/hse-coursework-user-sasrec

an embedding of item j. This can be seen as a matrix factorization model. When adding a new dimension (i.e., explicit user embedding) to the ranking score calculation, there are several possible approaches to consider. In [6] authors present a new framework for the task of collaborative next item prediction: a fusion of two different methods: Generalized Matrix Factorization (GMF) and Multi-Layer Perceptron (MLP). Both methods take user and item embeddings u and i respectively as input and output a score, which determines the way items are ranked for recommendation.

GMF aims to expand on the matrix factorization by generalizing the matrix factorization formula to

$$a_{ui} = a_{out}(h^\top(u \odot i))$$

Here h is the edge weights of the output layer and a_{out} is an activation function. For example, if we set a_{out} to identity and h to the vector of ones $\mathbb{1}$, we recover the original matrix factorization.

MLP aims to leverage the expressivity of deep neural networks by creating an MLP with its input being concatenated item and user vectors. Hidden layers allow for interactions between item and user latent features, thus increasing personalization and modeling collaborating filtering effect.

Inspired by [6], we consider two types of approaches to adding explicit user embeddings to SASRec: *concatenation-based* and *factorization-based*. Former aims to concatenate learned explicit user embedding to the item embeddings, and the latter changes the ranking score calculation inspired by matrix (tensor) decomposition.

Concatenation-based inclusion of explicit user embedding has already been studied in the Stochastic Shared Embeddings Personalized Transformer (SSE-PT) work [22]. There, user embeddings are concatenated to the item embeddings in the first layer of the SASRec model. It outperforms standard SASRec, but requires strong regularization in the form of Stochastic Shared Embeddings [23] to avoid overfitting.

In this work, we focus on factorization-based inclusion of explicit user embeddings by using several tensor decomposition methods instead, namely canonical (CP) decomposition [9] and Tucker decomposition [20].

We also compare our factorization-based approach with the additive approach that was proposed in the original SASRec paper, where it was reported to have no effect.

3 Methodology

Our main contribution is investigating the factorization-based inclusion of explicit user embedding. To this end, we consider two approaches: canonical decomposition and Tucker decomposition.

For the former, the final ranging score is calculated as

$$r_{ltj} = \sum_{k=1}^{D} u_{kl} h_{kt} i_{kj} = u_l^\top (h_t \odot i_j)$$

Note that this formula limits the user embedding size to be equal to the size of item (and thus, hidden state) embeddings.

For the latter, we calculate the score as

$$r_{ltj} = \sum_{\alpha=1}^{D_u} \sum_{\beta=1}^{D} \sum_{\gamma=1}^{D} u_{\alpha l} h_{\beta t} i_{\gamma j}$$

Here, D_u is the size of the user embedding. Note that Tucker decomposition allows us to change the size of user embeddings, potentially greatly reducing memory consumption in scenarios with large numbers of users.

For the canonical decomposition, we also consider several different ways of incorporating learned user embedding. We compare multiplication, multiplication with the squared user embeddings, which can be explained as learning weights for different latent dimensions (e.g., users' affinities for different genres) and GRU-like regularization, where we decide which components of the learned user embedding to include using gates from the user and hidden state embeddings.

4 Experimental setup

4.1 Datasets

We use several popular large datasets across different domains:

- Yambda-5B [17]. We use the 50m "listens" version of the dataset. To achieve the session-aware setting, similar to [15], the sequential dataset is split into sessions by 30-minute windows of inactivity.
- Gowalla [3]. Here, for the session-aware setting, we split similarly to Yambda, but with day-long inactivity windows.

- Yelp [2]. We don't introduce a session-aware version for this dataset, since it pans a substantial period of several years, and our inactivity window would have to have a magnitude of years.

For all datasets, consistent with prior research [16, 15], we filter items, users, and sessions with little interactions. More specifically, we exclude users with less than 20 interactions, items with less than 5 interactions, and session containing of less than 10 (20 for Yambda) interactions. Tables 4.1 and 4.2 contains dataset statistics for standard and session-aware settings correspondingly.

Table 4.1: Statistics of the datasets after preprocessing in the standard setting

Dataset	Users	Items	Interactions
Yambda	8,854	175,943	4,851,303
Gowalla	27,516	173,511	2,476,079
Yelp	42,429	137,039	2,284,489

Table 4.2: Statistics of the datasets after preprocessing in the session-aware setting

Dataset	Users	Items	Interactions	Number of sessions
Yambda	8,342	174,740	4,402,437	369,029
Gowalla	14,923	149,438	1,025,187	44,436

4.2 Evaluation

For all experiments, we use the global time point split, since it prevents time data leakages compared to the leave-one-out approach [10]. We split the data into training, validation, and test parts in a 90-5-5 proportion via corresponding quantiles. For the standard setting, we get the quantiles of all interactions, and for the session-aware setting, we get the quantiles of the means of session times, since it allows for a more equal distribution of sessions in dataset parts. Since we train both item and user embeddings, we drop test items and users that are not in the train set. Folding-in [4] of user embedding can be explored in further research. For evaluation, we follow the leave-one-out procedure of the test users or sessions. We do not use the information about the previous test sessions of the user and consider each session independently. We use the validation set for the hyperparameter tuning with the Optuna library [1] and for the control of model convergence via early stopping.

For performance analysis of the models, we use widely popular top-k recommendation metrics such as Normalized Discounted Cumulative Gain (NDCG@k), Hit Rate (HR@k) and

Coverage (COV@k) with $k = 10$. For further analysis we also split all items into 3 popularity buckets with equal number of interactions, and report NDCG@k metric for the items from the buckets: NDCG@k_{low}, NDCG@k_{mid}, NDCG@k_{high} where low is for least popular items and high – most popular.

We tuned the following hyperparameters: SASRec model hyperparameters (hidden size, number of heads in multihead attention, dropout rate, number of layers) and SCE hyperparameters: (number of buckets, y bucket size, mix operation). We used the same parameterization as in [16], so x bucket size is equal to the number of buckets. The final configurations can be found in the project repository[2]. All models were trained with Adam [12] optimizer with a constant learning rate equal to 10^{-3}.

5 Results

5.1 Standard setting performance comparison

We compare our method with the original SASRec model in the standard setting and summarize the results in Table 5.1.

Table 5.1: Results in the standard setting. The last column shows the relative improvement of our model over SASRec. Best results are in bold.

Dataset	Metrics	SASRec	UserSASRec	*Improv.*
Yambda	NDCG@10	**0.0431**	0.0402	−6.7%
	HR@10	**0.0640**	0.0605	−5.5%
	COV@10	**0.1732**	0.1137	−34.4%
	NDCG@10_{low}	**0.0515**	0.0439	−14.8%
	NDCG@10_{mid}	**0.0372**	0.0352	−5.4%
	NDCG@10_{high}	0.0376	**0.0388**	+3.2%
Gowalla	NDCG@10	0.0743	**0.0848**	+14.1%
	HR@10	0.1242	**0.1478**	+19.0%
	COV@10	0.2897	**0.4194**	+44.8%
	NDCG@10_{low}	0.0736	**0.0981**	+33.3%
	NDCG@10_{mid}	0.0610	**0.0642**	+5.2%
	NDCG@10_{high}	**0.0844**	0.0670	−20.6%
Yelp	NDCG@10	**0.0144**	0.0121	−16.0%
	HR@10	**0.0291**	0.0256	−12.0%
	COV@10	0.1680	**0.1978**	+17.7%
	NDCG@10_{low}	0.0054	**0.0058**	+7.4%
	NDCG@10_{mid}	**0.0120**	0.0117	−2.5%
	NDCG@10_{high}	**0.0317**	0.0232	−26.8%

They demonstrate that our method's relative performance strongly varies across datasets. While it substantially outperforms SASRec in certain scenarios, it also shows notable performance

[2] https://github.com/abramovgeorge/hse-coursework-user-sasrec

drops in others. This suggests that the model's effectiveness may be dataset-dependent, necessitating further evaluation across different datasets, which was not possible due to time constraints.

One notable change in the performance of our model is the quality of recommendations for low-popularity items. For the Gowalla dataset, NDCG@10_{low} has increased by one-third, while NDCG@10_{high} has decreased. For the Yelp dataset, NDCG@10_{low} is the only NDCG metric that has increased. We do not see this effect for the Yambda dataset, but for this dataset, low-popularity items exhibit better recommendation quality even for the basic SASRec, so our method may not be as effective. This effect can be seen in the metric dynamics during training in Figure 5.1.

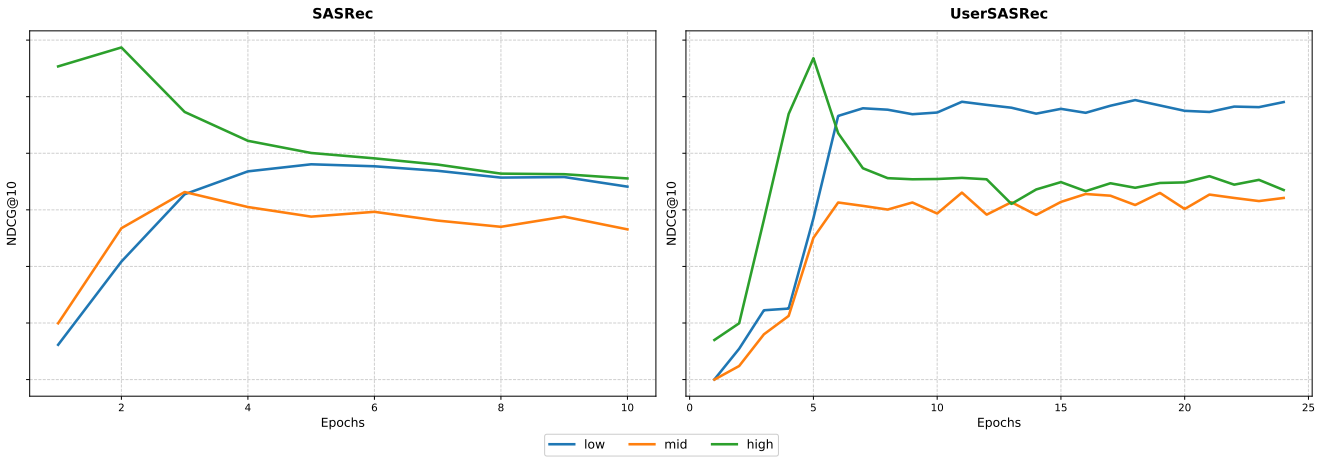

Figure 5.1: Relative NDCG@10 metric across different popularity bins for standard SASRec and our modification using the best configurations on the Gowalla dataset.

5.2 Session-aware setting performance comparison

We also provide the same comparison results but in the session-aware settings summarized in Table 5.2. We use the same configurations as in Table 5.1.

Table 5.2: Results in the session-aware setting. Best results are in bold.

Dataset	Metrics	SASRec	UserSASRec
Yambda	NDCG@10	**0.0597**	0.0480
	HR@10	**0.0946**	0.0833
	COV@10	**0.2960**	0.2146
	NDCG@10_{low}	**0.0830**	0.0636
	NDCG@10_{mid}	**0.0523**	0.0378
	NDCG@10_{high}	**0.0388**	0.0373
Gowalla	NDCG@10	**0.1034**	0.0912
	HR@10	**0.1680**	0.1461
	COV@10	0.0665	**0.0723**
	NDCG@10_{low}	**0.1139**	0.1066
	NDCG@10_{mid}	**0.1220**	0.1058
	NDCG@10_{high}	**0.0665**	0.0438

These results signify that the session-aware setting does not increase the performance of the model with extra user modeling, even when multiple sessions are associated with a single user. On the contrary, for the Gowalla dataset, SASRec starts to show supreme performance compared to the user modification. This may be explained by the fact that short sessions are not enough to form a cohesive and expressive user embedding, compared to the whole user histories.

We also note that since we only use full sessions for training and evaluation, global time point split and removal of the users not seen in train set from the test set results in a small testing set, which leads to biased evaluation. Possible solutions might include usage of split sessions in the test set, unknown user embedding folding-in, or foregoing the global time point split strategy entirely, as done in [15]. These approaches remain to be explored in future research.

5.3 Approaches for incorporating user embeddings for CP

Further, we explore different ways of incorporating user embeddings for the canonical decomposition. In this section, we also compare our tensor-based method with the addition-based method proposed in the original SASRec paper. Here, we did not tune each model on the validation set and used the best configuration from Table 5.1. We selected the Gowalla dataset among the three as it exhibits the most significant performance improvements for user embedding. Summarized results can be seen in Table 5.3.

Table 5.3: Results for the other methods of incorporating user embedding for the CP decomposition on the Gowalla dataset. Best results are in bold.

Dataset	Metrics	SASRec	Mult	Add	Mult^2	GRU
Gowalla	NDCG@10	0.0743	0.0848	0.0734	0.0798	**0.0950**
	HR@10	0.1242	0.1478	0.1272	0.1406	**0.1539**
	COV@10	0.2897	**0.4194**	0.3198	0.3652	0.3944
	$\text{NDCG@10}_{\text{low}}$	0.0736	0.0981	0.0767	0.0902	**0.1062**
	$\text{NDCG@10}_{\text{mid}}$	0.0610	0.0642	0.0604	0.0644	**0.0673**
	$\text{NDCG@10}_{\text{high}}$	0.0844	0.0670	0.0760	0.0648	**0.0867**

Here, several key observations can be made: First, addition-based user embeddings do not significantly improve the performance compared to basic SASRec, which was reported in [11]. Second, additional limitations on user embedding do not provide a significant increase in recommendation quality. We also note that the squared embeddings version requires approximately 5 times longer to converge compared to other models. And finally, we find that GRU-based regularization outperforms basic addition. It seems that complex architecture allows for greater control of user embedding influence over hidden state.

We can also note that these modifications increase the quality without needing to re-tune the hyperparameters.

5.4 Tucker decomposition

We also explore Tucker decomposition as a potential tensor formula for our final score. Here, we test it on 2 datasets in 2 different settings: for the Yambda dataset, we took the best model setup from Table 5.1 and changed hidden size form 128 to 64 due to time and memory constraints. And for the Gowalla dataset, we tuned the hyperparameters for the tucker decomposition with user embedding being equal to item embedding size. We present the results in Table 5.4.

Table 5.4: Results for the Tucker decomposition on the Yambda and Gowalla datasets. For the Yambda dataset, we used the best setup from Table 5.1 with the item embedding size reduced from 128 to 64. Best results are in bold.

Dataset	Metrics	CP	Tucker_{32}	Tucker_{64}	Tucker_{128}
Yambda	NDCG@10	**0.0300**	0.0067	0.0088	0.0103
	HR@10	**0.0493**	0.0137	0.0190	0.0208
	COV@10	0.0835	0.0628	0.0810	**0.0965**
	$\text{NDCG@10}_{\text{low}}$	**0.0311**	0.0028	0.0055	0.0075
	$\text{NDCG@10}_{\text{mid}}$	**0.0238**	0.0041	0.0056	0.0089
	$\text{NDCG@10}_{\text{high}}$	**0.0330**	0.0133	0.0138	0.0129
Gowalla	NDCG@10	0.0581	0.0704	**0.0914**	0.0804
	HR@10	0.1009	0.1134	**0.1518**	0.1358
	COV@10	0.2516	0.2101	0.3788	**0.3808**
	$\text{NDCG@10}_{\text{low}}$	0.0590	0.0714	**0.1057**	0.0958
	$\text{NDCG@10}_{\text{mid}}$	0.0402	0.0575	**0.0675**	0.0583
	$\text{NDCG@10}_{\text{high}}$	0.0694	**0.0757**	0.0735	0.0598

We can make two key observations from these results. First, Tucker decomposition can perform as well as canonical decomposition, which can be seen for the Gowalla dataset. Second, hyperparameters for CP and Tucker decompositions are completely different, and we cannot use the latter as a drop-in replacement. This follows from the results for the Yambda dataset, where the CP decomposition substantially outperforms the Tucker versions, and also for the Gowalla dataset, where the model with the canonical decomposition trained on the configuration for the Tucker decomposition significantly underperforms compared to the same model on the configuration for the CP decomposition.

Also, we note that increasing the user embedding for the Tucker decomposition does not necessarily increase the quality of recommendations (at least for the optimized configuration).

One possible explanation for these findings is SCE being derived from Maximum Inner Product Search (MIPS) problem, which implies logits being calculated as a dot product. The

behavior of Tucker decomposition when optimizing with SCE and other losses warrants a more thorough investigation.

6 Conclusion

In this work, we investigated several ways of introducing explicit user embeddings in the standard state-of-the-art SASRec model for sequential top-n recommendations.

Our experiments led to several key conclusions.

First, we demonstrated that the introduction of user embeddings can increase the quality of recommendations on some datasets. The observed large variance of relative performance across different datasets could be a focus for future research.

Second, our experiments show that explicit user modeling for SASRec tends to improve recommendation quality for low-popularity items. This suggests that this modeling allows for more specific and niche recommendations that arise from long-term user preferences.

Third, we find that the addition-based user modeling from the original SASRec paper indeed does not lead to a significant difference in performance compared to CP-based solutions. We also propose advanced methods for handling user embeddings for the canonical decomposition.

Finally, we explored Tucker decomposition as a cost-efficient alternative to CP decomposition. And while it shows competitive results, it remains sensitive to hyperparameters, with the model performance degrading with a decrease in user embedding size.

Acknowledgements

We thank the Higher School of Economics for granting access to the high-performance computing cluster "cHARISMa" [14] for conducting our experiments.

We also thank Petr Grinberg for providing the following PyTorch project template [5], which helped us implement the algorithms used in this paper by providing deep learning model training utilities, such as train/validation loops, dataset handling, metric logging, and more.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[2] Nabiha Asghar. "Yelp dataset challenge: Review rating prediction". In: *arXiv preprint arXiv:1605.05362* (2016).

[3] Eunjoon Cho, Seth A Myers, and Jure Leskovec. "Friendship and mobility: user movement in location-based social networks". In: *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2011, pp. 1082–1090.

[4] George W Furnas, Scott Deerwester, Susan T Durnais, Thomas K Landauer, Richard A Harshman, Lynn A Streeter, and Karen E Lochbaum. "Information retrieval using a singular value decomposition model of latent semantic structure". In: *Acm sigir forum*. Vol. 51. 2. ACM New York, NY, USA. 2017, pp. 90–105.

[5] Petr Grinberg. *PyTorch Project Template*. Sept. 2024. URL: https://github.com/Blinorot/pytorch_project_template.

[6] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. *Neural Collaborative Filtering*. 2017. arXiv: 1708.05031 [cs.IR]. URL: https://arxiv.org/abs/1708.05031.

[7] Xiangnan He, Hanwang Zhang, Min-Yen Kan, and Tat-Seng Chua. *Fast Matrix Factorization for Online Recommendation with Implicit Feedback*. 2017. arXiv: 1708.05024 [cs.IR]. URL: https://arxiv.org/abs/1708.05024.

[8] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. *Session-based Recommendations with Recurrent Neural Networks*. 2016. arXiv: 1511.06939 [cs.LG]. URL: https://arxiv.org/abs/1511.06939.

[9] Frank L Hitchcock. "The expression of a tensor or a polyadic as a sum of products". In: *Journal of Mathematics and Physics* 6.1-4 (1927), pp. 164–189.

[10] Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. "A critical study on data leakage in recommender system offline evaluation". In: *ACM Transactions on Information Systems* 41.3 (2023), pp. 1–27.

[11] Wang-Cheng Kang and Julian McAuley. *Self-Attentive Sequential Recommendation*. 2018. arXiv: 1808.09781 [cs.IR]. URL: https://arxiv.org/abs/1808.09781.

[12] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[13] Anton Klenitskiy and Alexey Vasilev. "Turning Dross Into Gold Loss: is BERT4Rec really better than SASRec?" In: *Proceedings of the 17th ACM Conference on Recommender Systems*. RecSys '23. ACM, Sept. 2023, pp. 1120–1125. DOI: 10.1145/3604915.3610644. URL: http://dx.doi.org/10.1145/3604915.3610644.

[14] PS Kostenetskiy, RA Chulkevich, and VI Kozyrev. "HPC resources of the higher school of economics". In: *Journal of Physics: Conference Series*. Vol. 1740. 1. IOP Publishing. 2021, p. 012050.

[15] Sara Latifi, Noemi Mauro, and Dietmar Jannach. "Session-aware recommendation: A surprising quest for the state-of-the-art". In: *Information Sciences* 573 (2021), pp. 291–315.

[16] Gleb Mezentsev, Danil Gusak, Ivan Oseledets, and Evgeny Frolov. "Scalable Cross-Entropy Loss for Sequential Recommendations with Large Item Catalogs". In: *18th ACM Conference on Recommender Systems*. RecSys '24. ACM, Oct. 2024, pp. 475–485. DOI: 10.1145/3640457.3688140. URL: http://dx.doi.org/10.1145/3640457.3688140.

[17] A. Ploshkin, V. Tytskiy, A. Pismenny, V. Baikalov, E. Taychinov, A. Permiakov, D. Burlakov, E. Krofto, and N. Savushkin. *Yambda-5B – A Large-Scale Multi-modal Dataset for Ranking And Retrieval*. 2025. arXiv: 2505.22238 [cs.IR]. URL: https://arxiv.org/abs/2505.22238.

[18] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. *Improving language understanding by generative pre-training*. 2018.

[19] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. *BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer*. 2019. arXiv: 1904.06690 [cs.IR]. URL: https://arxiv.org/abs/1904.06690.

[20] Ledyard R Tucker. "Some mathematical notes on three-mode factor analysis". In: *Psychometrika* 31.3 (1966), pp. 279–311.

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[22] Liwei Wu, Shuqing Li, Cho-Jui Hsieh, and James Sharpnack. "SSE-PT: Sequential recommendation via personalized transformer". In: *Proceedings of the 14th ACM conference on recommender systems*. 2020, pp. 328–337.

[23] Liwei Wu, Shuqing Li, Cho-Jui Hsieh, and James Sharpnack. *Stochastic Shared Embeddings: Data-driven Regularization of Embedding Layers*. 2020. arXiv: 1905.10630 [cs.LG]. URL: https://arxiv.org/abs/1905.10630.